

May 31 - Build Agentic AI Systems That Actually Ship. 2026-06-05 19:30(GMT-7:00)

 Manual notes  Transcript  Workflow

Key Outcomes

Session 1 established production-ready LLM foundations, moving beyond demo chatbots to systems with memory, error handling, and structured logging. Learners built a customer support agent with 3-turn conversation memory, implemented graceful error degradation, and logged latency/token usage in JSON format. [1](#) [2](#) [3](#)

Session Overview

Core Mental Model:

- **Agentic systems** use a decision loop (observe → think → act → repeat), not single LLM calls [4](#) [5](#) [6](#)
- **Production risks** include infinite loops, cost explosion, hallucination, wrong tool calls, bad memory, no audit trail, unsafe actions, and slow response [7](#) [8](#)
- **Agentic anatomy** has 4 components: reasoning engine (LLM), memory/state (conversation history, tool results, user context), planning/routing (decision logic), and tools/actions (deterministic functions) [8](#) [9](#) [10](#)

Key Principles:

- **LLMs are stateless** – models don't own memory; applications must provide context via message history [8](#) [11](#) [12](#)
- **LLM proposes, code disposes** – LLMs suggest actions (e.g., refund \$500), application code validates and executes deterministically [13](#) [14](#) [15](#)
- **Temperature is an engineering decision** – use 0–0.3 for support/compliance/tool calls

(reliability), 0.5–0.7 for balanced conversation, 0.8–1.0 for creative writing 16 17 18

Technical Setup Completed

Environment Configuration:

- Installed Anaconda distribution and VS Code/Cursor IDE 1 19
- Created `.env` file with OpenAI API key and Google Gemini API key (free tier available) 20 21 22
- Set up virtual environment using `conda create --prefix ./env python=3.12` and activated with `conda activate ./env` 23
- Installed dependencies via `pip install -r requirements.txt` (LangChain, LangGraph, OpenAI, Google GenAI libraries) 24
- Created `.gitignore` to exclude `.env`, `__pycache__`, and `./env` from version control 25

Framework Rationale:

- **LangChain framework** enables model-agnostic code – switching from OpenAI to Google Gemini requires only changing LLM initialization, not entire application logic 26 27 28
- Framework provides pre-built patterns for loops, memory, tools, and RAG instead of building from scratch 29 30

Memory and Messages Implementation

Message Types:

- **System message:** Defines personality, rules, constraints (e.g., "You are a helpful assistant that responds in a funny manner") 31 32
- **Human message:** User input 31
- **AI message:** Model's response 31

- **Tool message:** Output from tool execution (mentioned for agentic systems) 33

Memory Pattern:

- Conversation stored as Python list of typed messages from `langchain_core.messages` 31 34
- Each new turn appends human message → invokes LLM → appends AI response → sends full history on next call 34 35
- **Cost implication:** More context increases input tokens (example: 36 → 63 input tokens when adding conversation history) 36 37

Hands-On Activity:

- Learners built 3-turn customer support conversation with visible context continuity (e.g., agent remembers issue from turn 1 when asked "Do you remember my issue?" in turn 3) 38 39

Production-Aware Patterns

Error Handling (`production_invoke` wrapper):

- Wrapped LLM calls in try-except blocks catching `APIError` , `ValueError` , `TimeoutError` , and general exceptions 40 41
- Validates non-empty responses before returning 40
- Returns graceful degradation messages (e.g., "Something went wrong, our team has been notified") instead of raw error traces 40 42

Structured Logging:

- Logs in **JSON format** (queryable by CloudWatch, Datadog, Grafana) instead of print statements 43 44
- Captures: event type (`LLM_response`), session ID (trace ID from frontend), latency (milliseconds), input/output/total tokens, error type 45 46 47
- Enables queries like "show all calls over 3000ms" or "error rate for session X" 48

Usage Metadata Tracking:

- Every LLM response includes `usage_metadata` with input tokens, output tokens, total tokens, reasoning tokens 49 50
- **Critical for cost tracking** – "a system without token logging is running blind" 50

Production Readiness Checklist

Day 1 Foundations (covered):

- Error handling for LLM call failures 15 51
- Structured logging with latency and token usage 15 51
- Loop protection via maximum iteration limits (5–10 steps typical) 52 53
- Secrets management (`.env` file, not in code/git) 15 25

Future Topics (Weeks 2–8):

- Cost tracking per session 15
- Graceful degradation with fallback models 54 55
- Input validation against crafted prompts 54
- Timeout handling for external tool calls 54

Key Technical Observations

Model Behavior:

- Without conversation history, asking "What is his age?" after "Who is PM of India?" fails – model responds "Who are you referring to?" 49 56
- With message history, model correctly infers "his" refers to previously mentioned person 35 36
- Output varies by model provider (OpenAI GPT-4o vs Google Gemini 2.0 Flash) but core behavior (statelessness) is consistent 57 58

API Differences:

- OpenAI API: `client.responses.create()` with `model` parameter 59 60
- Google GenAI: `genai.models.generate_content()` with different structure 60
- LangChain abstracts these differences via `ChatOpenAI` and `ChatGoogleGenerativeAI` classes 28

Security and Authorization

Agent Security Principles:

- Prevent unauthorized tool calls via policy documents (covered in future sessions) 61 62
- Validate user permissions before executing external API calls (e.g., ServiceNow) 62
- Never execute tool proposals without application-level validation 14 15

Hallucination Detection:

- Validate tool calls against allowed actions 63 64
- Use golden datasets for output validation 63
- Apply rule-based checks on final outputs 64

Learner Ownership Block

In-Class Activity:

- Built 3-turn customer support conversation with memory using system/human/AI messages 38
- Implemented error handling wrapper for LLM calls 41
- Added structured JSON logging with latency and token tracking 46 47

Proof of Progress Assignment:

- Deliverable: Support agent with configurable persona, 4-turn conversation, message memory, error handling, structured logging, clean terminal output 65

- Required reflection (5 lines): What did you build? Where is memory handled? Where can this fail? What makes it production-aware? What will you improve next? ⁶⁶
- Submission: Include reflection in README and share in WhatsApp group ⁶⁶

Repository Access

GitHub Setup:

- Learners fill form with name, enrollment email, **exact GitHub username** (case-sensitive), and cohort (May 31) ⁶⁷ ⁶⁸
- Invitation sent automatically within minutes (max 6–8 hours) ⁶⁸ ⁶⁹
- Repository contains live code updated during sessions; local zip archives added to portal after completion ⁷⁰

Next Steps

- **Session 2:** Tomorrow, 8 AM IST ⁷¹
 - **Office Hours:** Wednesdays, 9:30 PM IST for Q&A ⁷²
 - **Assignment Deadline:** Instructions shared via email and WhatsApp within 3–4 hours ⁶⁵
 - **Action Items:** Complete proof-of-progress build, submit with 5-line reflection, commit to "iCommit" pledge ⁶⁶ ⁷³
-